

Ensembles and random forests

Dataset: Forest CoverType

How to use these notes

The primary text for this lecture is Géron, Chapter 6. These notes are not a replacement for it and do not repeat it: they are the lecture’s own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture’s notebook, and the course checks that mechanically rather than trusting it.

Contents

1	The result we are carrying forward	2
1.1	Where many different models come from	2
2	Derivation: the variance of an average of correlated predictors	3
2.1	Two, then n	3
2.2	Reading the two terms	4
3	Building them, and measuring ρ	4
3.1	A validation set you get for nothing	4
3.2	Measuring ρ needs something we do not have	5
3.3	Does the formula actually hold?	5
3.4	Does each variant lower ρ ?	6
4	And what did that do to the accuracy?	6
5	The bill	7
5.1	Feature importance, and a control	8

6	Boosting, and what practitioners reach for	9
7	Did the averaging fix what we diagnosed?	10
8	The notebook	10
9	Exercises	10
	Summary	11

1 The result we are carrying forward

Lecture 6 built a decision tree on this dataset and ended with a measurement rather than a score. Refit the same depth-8 tree, with the same hyperparameters and the same seed, on twenty different 90% subsamples of the same training rows: test accuracy moves by a standard deviation of 0.25 points, and *9.1% of the individual predictions change*. Only 71.4% of test patches get the same answer from all twenty.

A stable metric, an unstable model. In the language of Lecture 5's decomposition this is *variance*: sensitivity of the fitted function to the particular training sample. The whole of today follows from one question: what does averaging do to variance?

Forest CoverType	Test accuracy	Conditions per justification
Always "Lodgepole Pine"	48.8%	0
Depth-8 tree, minimum leaf 20	73.0%	7.97
Unconstrained tree	82.6%	17.88

1.1 Where many different models come from

CART is deterministic: same data, same tree. To get twenty different trees you must give them twenty different things to look at.

Source of difference	Name
Different training algorithms on the same data	a voting classifier
The same algorithm on different random subsets of <i>rows</i> , drawn with replacement	bagging
... drawn without replacement	pasting
... plus random subsets of <i>columns</i> at each split	random forest
... plus random <i>thresholds</i>	extra-trees

The list is in increasing order of how hard it works to make two members unlike each other. That ordering is not an accident.

Why this lecture follows the one on trees: averaging attacks variance, and trees are the high-variance model we have just measured; a tree is cheap to fit, so two hundred are affordable; and an unconstrained tree has *low bias* — which matters, because averaging does nothing whatever to bias, so the members had better not have any to spare. That third point is why the ensembles here use unconstrained members. It looks reckless after a lecture spent limiting depth, and by the end of the next section it will look obvious.

2 Derivation: the variance of an average of correlated predictors

Forget trees for ten minutes. You have n predictors of the same quantity, each with variance σ^2 , and you average them: $\bar{f} = \frac{1}{n} \sum_t f_t$. What is $\text{Var}(\bar{f})$?

The answer everyone gives first is σ^2/n . That step is valid only when the f_t are *uncorrelated* — the variance of a sum is the sum of the variances *plus twice the sum of the covariances*. Dropping the covariances is an assumption, and it is exactly the assumption that fails for an ensemble of trees grown on the same dataset.

2.1 Two, then n

With $\text{Var}(f_1) = \text{Var}(f_2) = \sigma^2$ and $\text{Corr}(f_1, f_2) = \rho$:

$$\text{Var}\left(\frac{f_1+f_2}{2}\right) = \frac{1}{4}[\sigma^2 + \sigma^2 + 2\rho\sigma^2] = \frac{\sigma^2(1+\rho)}{2}.$$

At $\rho = 0$ this is the familiar halving; at $\rho = 1$ it is σ^2 , because the two predictors are the same predictor and averaging achieves nothing. The general case has both limits in it.

n identically distributed predictors, common pairwise ρ

Assume each has variance σ^2 and $\text{Cov}(f_s, f_t) = \rho\sigma^2$ for $s \neq t$. Then

$$\text{Var}\left(\sum_{t=1}^n f_t\right) = \sum_{s=1}^n \sum_{t=1}^n \text{Cov}(f_s, f_t),$$

a double sum of n^2 terms. Split them by whether $s = t$: n diagonal terms each σ^2 , and $n(n-1)$ off-diagonal terms each $\rho\sigma^2$. Divide by n^2 :

$$\text{Var}(\bar{f}) = \frac{1}{n^2}[n\sigma^2 + n(n-1)\rho\sigma^2] = \frac{\sigma^2}{n} + \frac{n-1}{n}\rho\sigma^2,$$

and writing $\frac{n-1}{n} = 1 - \frac{1}{n}$ and gathering the $1/n$ terms,

$$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{n}$$

Identically distributed, not independent — that is the whole point. n trees grown on bootstrap samples of one dataset have the same distribution and are certainly not independent: they were handed the same dataset.

2.2 Reading the two terms

Term	Depends on n ?	What it is
$\rho\sigma^2$	no	the part every predictor shares
$(1 - \rho)\sigma^2/n$	yes, like $1/n$	the part each predictor has to itself

The sentence to remember

Averaging destroys the independent component of the variance and leaves the correlated one completely untouched.

And note what it does *not* say. It says nothing about bias: an average of n biased predictors is exactly as biased as one of them.

Three limits are worth memorising. At $\rho = 0$ the average has variance σ^2/n , the classical result. At $\rho = 1$ it has variance σ^2 and averaging is a no-op. As $n \rightarrow \infty$ it tends to $\rho\sigma^2$, the floor — and because the second term falls like $1/n$, that floor is reached quickly: going from 1 member to 10 removes 90% of the independent part, while going from 100 to 1,000 removes 0.9% more.

So the way to improve an ensemble is *not more members*. It is less correlated members. And — the point that gets forgotten — anything which lowers ρ by making individual members worse is a trade, not a free lunch, because σ^2 appears in *both* terms and can rise while ρ falls.

One formula, three algorithms

Bagging randomises the rows. A random forest also randomises the columns. Extra-trees also randomises the thresholds. All three exist for one purpose: to lower ρ , because $\rho\sigma^2$ is the floor that n cannot reach below.

3 Building them, and measuring ρ

Bagging draws m rows with replacement, fits a tree, and repeats. **Pasting** is the same without replacement, producing less diverse members. A **random forest** adds a random subset of features at every node — the classification default `max_features="sqrt"` gives $\lfloor \sqrt{54} \rfloor = 7$ of our 54 columns, drawn afresh at each node, so a tree with 200 nodes draws 200 different subsets. **Extra-trees** goes one step further: having chosen the candidate features, draw one threshold at random for each rather than searching for the best. Note that Scikit-Learn's extra-trees default is `bootstrap=False`, so the random thresholds *replace* the bootstrap rather than joining it.

3.1 A validation set you get for nothing

Each bagged member never sees about 37% of the rows — its **out-of-bag** instances. A given row is missed by one draw with probability $1 - 1/m$, so by all m draws with probability

$$\left(1 - \frac{1}{m}\right)^m \rightarrow e^{-1} \approx 0.368,$$

hence about 63% distinct rows in and 37% held out. Score each member on its own out-of-bag rows and you have a generalisation estimate with no validation split and no extra fit.

Read it carefully, though. It is an average over members, each scored on a different 37%, so it estimates the generalisation of one member's *procedure*, not of the ensemble you ship. It is available only where members see bootstrap samples. And it does not replace the test set — it replaces the validation set.

A leakage detector you get free

If the out-of-bag score comes out much *better* than the test score, something in the pipeline saw the out-of-bag rows anyway — a scaler fitted outside the ensemble, or a feature built from the whole frame.

3.2 Measuring ρ needs something we do not have

ρ is the correlation between two members over the randomness of the whole procedure, which includes *which training set you were handed*. Two trees grown on one dataset are correlated through that dataset — so a single training set cannot show it. Conditional on the data, the members are independent by construction and ρ would measure as zero.

CoverType has 581,012 rows, so we can cut 20 *disjoint* training sets of 20,000 rows each — 400,000 rows, no row reused — and still keep 12,000 aside as a shared test set. At each test patch, decompose the variance of a single member across all 400 fitted trees:

$$\sigma^2 = \underbrace{\tau^2}_{\text{between training sets}} + \underbrace{w^2}_{\text{within a training set}}, \quad \rho = \frac{\tau^2}{\tau^2 + w^2}.$$

τ^2 is the part every member of a given ensemble inherits from its data, and it *is* the floor $\rho\sigma^2$ — not a coincidence, but the same quantity written twice.

Failure condition — one trap in that code, worth ten minutes of your life

```
>>> rnd.classes_          # the ensemble's labels
array([1, 2, 3, 4, 5, 6, 7])
>>> rnd.estimators_[0].classes_  # one member's labels
array([0., 1., 2., 3., 4., 5., 6.])
```

Every ensemble re-encodes the labels as *positions* before handing them to its members, so a member's `predict` returns positions, not cover types. Compare them with `y_test` directly and every member scores about 8% — which looks like a terrible model rather than a bug.

3.3 Does the formula actually hold?

Ensemble	σ^2	ρ	Var of the average of 20	... predicted
Bagging	0.1542	0.072	0.01814	0.01819
Random forest	0.1675	0.043	0.01508	0.01523
Extra-trees	0.1648	0.067	0.01859	0.01867
Extra-trees, bootstrap=True	0.1735	0.043	0.01554	0.01570

Measured and predicted agree in the third decimal place on all four. The derivation is a description

of this dataset. Agreement this good is partly luck — the formula assumes every pair has the same ρ , which is only approximately true — so do not expect three decimals every time.

3.4 Does each variant lower ρ ?

Ensemble	Rows	Columns / thresholds	ρ
Bagging	bootstrap	all 54, best threshold	0.072
Extra-trees (default)	all	7 of 54, random threshold	0.067
Random forest	bootstrap	7 of 54, best threshold	0.043
Extra-trees, bootstrap=True	bootstrap	7 of 54, random	0.043

Every variant is below bagging, so the claim survives. But read the last two rows: adding random thresholds on top of a random forest moves ρ from 0.043 to 0.043. *The mechanisms are not additive.*

Feature subsampling is doing the work, cutting ρ by 40%. Random thresholds are a *substitute* for the bootstrap rather than an addition to it — extra-trees without a bootstrap lands at 0.067, near bagging; with one, exactly where the forest is. And there is a floor to the floor: nothing we tried gets below 0.043, because all four ensembles ultimately saw the same 20,000 rows.

The formula's real message

ρ is not a property of the algorithm. It is the share of the variance that comes from *which data you were given*, and no amount of algorithmic randomness reaches it.

Ensemble	One member	Average of 20	Removed	Floor $\rho\sigma^2$
Bagging	0.15453	0.01814	88.3%	0.01103
Random forest	0.16734	0.01508	91.0%	0.00721
Extra-trees	0.16439	0.01859	88.7%	0.01098
Extra-trees, boot	0.17248	0.01554	91.0%	0.00740

Twenty members remove about nine tenths of a single member's variance. Two hundred would remove roughly one twentieth more, and then stop — exactly what $(1 - \rho)\sigma^2/n$ says.

4 And what did that do to the accuracy?

200 members, 48,000 training patches	10 members	200 members
Bagging	87.7%	89.8%
Extra-trees	86.7%	88.9%
Random forest	85.9%	88.4%
Extra-trees, bootstrap=True	85.2%	88.0%
The legible depth-8 tree	—	73.0%
Always “Lodgepole Pine”	—	48.8%

Every ensemble beats the constrained tree by at least fifteen points and the unconstrained tree by at least five. The last 190 members buy about two.

Now put the two tables beside each other:

Ensemble	ρ	One member's accuracy	Ensemble accuracy
Bagging	0.072	78.8%	89.8%
Extra-trees	0.067	77.8%	88.9%
Random forest	0.043	73.6%	88.4%
Extra-trees, boot	0.043	72.8%	88.0%

The ensemble with the highest ρ is the most accurate.

That is not a contradiction — read the formula again

The formula is about *variance*. It says nothing about bias, and accuracy depends on both. Every mechanism that lowers ρ does so by *handicapping the members*. A tree allowed only 7 of 54 columns per node is a worse tree: 73.6% against bagging's 78.8%. And σ^2 appears in both terms, so lowering ρ while raising σ^2 can leave you worse off.

On this dataset — 54 columns of which perhaps a dozen carry most of the signal — throwing away 47 at every node costs more than the decorrelation is worth. On a dataset with hundreds of weakly informative columns it is usually the other way round, which is why the default exists.

What the derivation predicted, and what it declined to

It predicted the shape of the accuracy curve, the variance of a 20-member average to three decimals on four different ensembles, and that more members would stop helping and roughly when. It did *not* tell us which ensemble would be most accurate, and it never claimed to.

A theory that predicts three things and declines to predict the fourth is not a weak theory. A theory that predicts all four without being asked is usually being misread.

5 The bill

Failure condition — an ensemble has no per-prediction justification

The 200 trees hold **1,497,919 leaves** between them. The justification for one prediction is now 200 decision paths and a vote.

The agency asked for a model whose every prediction comes with a human-readable justification. We have built one that is 15.4 points more accurate and cannot supply one. *There is no technical fix*. The tools below recover something, and what they recover is genuinely less than what was lost. Saying so is part of the job.

The agency asked for	An ensemble can offer
Why this parcel	nothing comparable
Which measurements the decision rests on, overall	feature importance
How confident the model is	the vote share, better calibrated than one leaf
Reproducibility of the answer	much improved — that was the whole repair

Three of four is not nothing. The one we cannot supply is the one the agency wrote down first.

On the vote share: a single tree's `predict_proba` returns the class proportions of one leaf, piecewise constant and often built on a handful of rows. An ensemble averages 200 such vectors from 200 different leaves, so the result varies smoothly and is better calibrated — the one item on the agency's list that averaging improves rather than destroys. But better calibrated is not calibrated: a forest's vote share is systematically pulled towards the middle, because unanimity is rare.

5.1 Feature importance, and a control

`feature_importances_sums`, over every node that split on a feature, the weighted impurity reduction that split achieved — the ΔI of Lecture 6, accumulated. It runs, it is fast, and the top of the list is entirely sensible: elevation first, then the distances. Nothing invites suspicion.

Before trusting any ranking

Ask what it would look like on a variable you know carries no information. If you cannot answer, add one and find out.

Add a column of uniform random numbers and refit. It ranks **tenth of fifty-five** — above 45 of the 54 real measurements, with an impurity importance of 0.0399 against 0.2277 for elevation. Put that list in front of the agency and you have asserted that a column of noise matters more than 45 things the surveyors actually recorded.

Failure condition — two structural biases, not an accident

Impurity importance is measured on the *training* data, and a split that reduces impurity on the rows that chose it will do so whether or not the feature is informative.

And it favours *high-cardinality* features: a continuous column offers thousands of candidate thresholds where a 0/1 soil-type column offers one, so more chances to find a lucky split means more accumulated ΔI . Our decoy is continuous and 44 of the 54 real columns are binary.

The repair is to break the column and see what breaks: shuffle one column of the *held-out* set, re-score, and report a spread over repeats.

Measurement	random_decoy	Elevation
Impurity importance (training data)	0.0399	0.2277
Rank among the 55 columns, by impurity	10th	1st
Permutation importance (held out)	$+0.00085 \pm 0.00195$	far above it

Permutation importance puts the decoy at less than half a standard deviation from zero — the correct answer, which the impurity measure never had any way of reaching. And the forest with the decoy scores 87.6% against 88.4% without: adding a noise column costs a little accuracy and a lot of credibility.

Question	Can importance answer it?
Which measurements should the survey keep collecting?	yes — this is what it is for
Is the model using a variable it legally must not?	yes, as a screen
Why was this parcel refused?	no
Would removing this column hurt?	no — remove it, refit, and measure

Row three is the agency's actual question, and it is the row importance cannot fill. Do not present it as though it can.

6 Boosting, and what practitioners reach for

Everything so far trains members in parallel and averages them. **Boosting** trains them in *sequence*, each correcting its predecessor. The members are not identically distributed and are certainly not exchangeable, so *today's formula does not apply to boosting*. Bagging reduces variance; boosting reduces bias. Two consequences follow from the sequence: boosting cannot be parallelised across members, and it is much easier to overfit with.

AdaBoost fits a weak learner — typically a depth-1 stump — increases the weight of the instances it got wrong, and repeats, predicting by a weighted vote. **Gradient boosting** keeps the sequence but changes the correction: each new learner is fitted to the *residual errors* of the ensemble so far, with `learning_rate` scaling each contribution.

Failure condition — the intuition that does not carry across

Adding trees to a bagged ensemble can only help, because $(1 - \rho)\sigma^2/n$ falls monotonically. Adding trees to a *boosted* ensemble eventually hurts. Carrying the first intuition into the second is one of the commonest mistakes with these models.

`HistGradientBoostingClassifier` bins each feature into at most 255 buckets before growing anything, so a split search costs $O(\text{bins})$ rather than $O(m)$. It is the algorithm `LightGBM` and `XGBoost` implement, handles missing values and categorical columns natively, and on tabular data is the usual first choice. On this dataset, at a budget that fits a lecture, it lands several points below the bagged forest — usual is not always.

Stacking trains a model to do the aggregation, with the base learners' predictions as the input features of a *blender*. The blender must be trained on predictions the base learners made for instances they did not see; `StackingClassifier` does this with cross-validation internally, which is the only reason it is safe to use. Build it by hand and the blender sees in-sample predictions, learns to trust whichever base learner overfits hardest, and the ensemble comes out worse than its best member — with no error and no warning.

Situation	Reach for
A high-variance base learner and time to spare	bagging or a random forest
Many weakly informative columns	a random forest — feature subsampling earns its keep
Training time is the binding constraint	extra-trees
You want the strongest tabular model	histogram gradient boosting — usually, and measure it against a forest anyway
You must justify individual predictions	none of them — go back to one tree

The last row is what this pair of lectures exists to teach, and it is the row people skip.

7 Did the averaging fix what we diagnosed?

The diagnosis was *instability*, not inaccuracy — so test the thing that was diagnosed, not the thing that is easy to measure. On 20 disjoint training sets, a random forest’s variance of one member’s correctness is 0.16734 and of the average of 20 is 0.01508: 91.0% of the variance removed, against a floor of 0.00721 it cannot reach below. Nine tenths of a single member’s variance, gone — and we knew before starting both that it would stop and roughly where.

The standing constraint, extended by two clauses

Split before anything is fitted. All preprocessing inside a Pipeline passed to cross-validation. Nothing derived from the test set in the training path. Fixed seeds. Per-fold scores, not just the mean. **Any importance or explanation is computed on held-out data and reported with a control. Any per-instance claim is checked for stability under refitting.** Both new clauses come from a measurement in this lecture rather than from a principle: the decoy at rank 10 of 55, and 9.1% of predictions moving while the accuracy did not.

8 The notebook

What to change

Set `max_features=None` on the random forest, which turns it back into bagging. Watch ρ rise, each member get better, and the ensemble accuracy rise with it — the trade of the whole lecture, in one keyword.

9 Exercises

- For n predictors each of variance σ^2 and pairwise correlation ρ , the variance of their average is $\rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$. State the limit as $n \rightarrow \infty$, and what it means for ensembles. [5]
- Extra-trees are usually faster to fit than a random forest and often no less accurate. Give the mechanism, in terms of the formula above. [4]

3. You set `bootstrap=False` and `oob_score=True`. State what happens and why. [3]
4. Impurity-based feature importance ranks a random decoy column above several real ones. Explain how that can happen, and name the measure you would use instead. [5]
5. A forest improves accuracy over one tree by far less than the variance reduction would suggest. Is this a contradiction? Answer with a reason. [4]

Summary

From the double sum of covariances, $\text{Var}(\bar{f}) = \rho\sigma^2 + (1 - \rho)\sigma^2/n$: averaging destroys the independent component of the variance and cannot touch the correlated one. Read as a policy, that says more members buy less and less, and the way to improve an ensemble is less correlated members.

Measured on 20 disjoint training sets, the formula predicted the variance of a 20-member average to three decimals on four different ensembles. ρ fell from 0.072 to 0.043 — and the mechanisms turned out not to be additive: random thresholds substitute for the bootstrap rather than adding to it, and nothing reached below 0.043 because all four ensembles saw the same 20,000 rows. ρ is not a property of the algorithm but of which data you were given.

The lowest- ρ ensemble was not the most accurate, because every mechanism that lowers ρ also weakens the members and σ^2 sits in both terms. And the 15.4 points of accuracy cost 1,497,919 leaves and the per-prediction justification the previous lecture was built around. Impurity importance ranked a column of pure noise 10th of 55; a permutation measurement on held-out data put it half a standard deviation from zero, where it belongs.